

Module 7 — Microservices

Highest priority for senior roles. Interviewers care less about “what is a microservice” and more about **resilience patterns, distributed data (saga/outbox), idempotency, and communication trade-offs**. Bring concrete failure scenarios.

7.1 Monolith vs Microservices

Core Concept

- **Monolith** — one deployable, one codebase, one DB. Simple to build/deploy/ debug; strong consistency; scales as a whole.
- **Microservices** — many small, independently deployable services, each owning its data, communicating over the network.

Aspect	Monolith	Microservices
Deploy	one unit	independent per service
Scaling	whole app	per service
Data	shared DB, ACID	DB-per-service, eventual consistency
Team	coupled	autonomous teams
Complexity	low ops, high code coupling	high ops (network, tracing, saga)
Failure	in-process	partial failures, network

Trade-offs / Common Mistakes

Microservices trade code complexity for **distributed-systems complexity** (network, partial failure, data consistency, observability). Anti-patterns: distributed monolith (services deploy together / share a DB), nano-services, shared database across services. **Start with a modular monolith**; extract services when scaling/team boundaries demand it.

Interview Q / Follow-ups

- When would you NOT use microservices?
- What is a distributed monolith and why is it bad?
- How do you decide service boundaries? (*bounded contexts / DDD.*)

7.2 API Gateway

Core Concept

Single entry point for clients. Handles cross-cutting concerns: routing, auth/token validation, rate limiting, request aggregation, TLS termination, CORS, load balancing. Spring Cloud Gateway (reactive) is the common choice.

ASCII

```
clients -> [ API GATEWAY ] (auth, rate-limit, routing, aggregation)
          |-> order-service
          |-> user-service
          |-> payment-service
```

Trade-offs

Central point of failure/latency → run HA, keep logic thin. Distinguish from a **service mesh** (sidecar, east-west traffic, mTLS) vs gateway (north-south).

Interview Q

Gateway responsibilities; gateway vs load balancer vs service mesh; BFF pattern.

7.3 Service Discovery & Config Server

Service Discovery

Services register with a registry (Eureka, Consul, or Kubernetes DNS/Services) so callers resolve instances dynamically instead of hardcoding hosts. - **Client-side** (Eureka + Ribbon/Spring Cloud LoadBalancer): client picks an instance. - **Server-side** (K8s Service, cloud LB): infra routes.

Config Server

Centralized, versioned external config (Spring Cloud Config backed by Git); services fetch config at startup; refresh via `@RefreshScope` / bus. In K8s, ConfigMaps/Secrets often replace it.

ASCII

```
service starts -> register in Eureka -> heartbeat
caller -> discovery -> instance list -> load-balance -> call
```

Interview Q

Client vs server-side discovery; how config changes propagate; secrets handling.

7.4 Resilience: Circuit Breaker, Retry, Timeout, Bulkhead

Why Interviewers Ask This

Partial failure is the defining trait of distributed systems. They want the four core patterns and how they interact (Resilience4j in Spring).

Patterns

- **Timeout** — never wait forever; bound every remote call. *First line of defense.*
- **Retry** — retry transient failures, with **exponential backoff + jitter**; only for **idempotent** ops; cap attempts.
- **Circuit Breaker** — stop calling a failing dependency; states **CLOSED** → **(failure rate exceeds threshold)** → **OPEN** → **(after wait)** **HALF_OPEN** → **CLOSED**. Fails fast + fallback while OPEN, giving the dependency time to recover.
- **Bulkhead** — isolate resources (separate thread pools / concurrency limits per dependency) so one slow dependency can't consume all threads (like ship compartments).
- **Rate limiter / fallback** round out the toolkit.

ASCII — Circuit Breaker States

```

      failures >= threshold
CLOSED  ───────────────────> OPEN
      |                         |
      | success in HALF_OPEN    | wait duration
      |                         |
      |                         v
      | ─────────── HALF_OPEN ──┘ trial request
      | (failure -> back to OPEN)

```

Real Production Example (Resilience4j)

```
@CircuitBreaker(name = "pricing", fallbackMethod = "cachedPrice")
@Retry(name = "pricing")
@TimeLimiter(name = "pricing")
public CompletableFuture<Price> getPrice(String sku) { ... }

Price cachedPrice(String sku, Throwable t) { return cache.getLast(sku); }
```

Common Mistakes / Trade-offs

- Retrying non-idempotent operations (double charge).
- Retry storms amplifying an outage (no backoff/jitter, no circuit breaker).
- Timeouts longer than the caller's timeout (cascading).
- No fallback → circuit breaker just fails fast with errors.

Interview Q / Follow-ups

- Explain circuit breaker states.
- Why combine retry + circuit breaker + timeout? Order of interaction?
- Why must retried operations be idempotent?
- What is a bulkhead and what problem does it solve?
- What is a retry storm / thundering herd; how to prevent (backoff + jitter)?

Hands-on Exercise

Wrap a flaky client with Resilience4j `@Retry` (exp backoff+jitter) + `@CircuitBreaker` + fallback; trip the breaker and observe `HALF_OPEN` recovery.

7.5 Distributed Transactions, Saga & Outbox

Why Interviewers Ask This

DB-per-service kills 2PC/XA in practice. They want the **saga** pattern and how to publish events reliably (**outbox**).

The problem

No single ACID transaction spans services. 2PC (XA) is blocking, doesn't scale, and couples services. Instead use **eventual consistency** with sagas.

Saga Pattern

A saga = a sequence of local transactions, each publishing an event that triggers the next; failures trigger **compensating transactions** (semantic undo). - **Choreography** — services react to each other's events (no central coordinator). Simple, but logic is spread out; hard to track. - **Orchestration** — a central orchestrator tells each service what to do and handles compensation. Clearer, testable; orchestrator is a dependency.

ASCII — Order Saga (orchestration)

```
Orchestrator: create order (pending)
-> reserve inventory (ok?)
   -> charge payment (ok?)
       -> confirm order
           (payment fails) -> compensate: release inventory -> cancel order
```

Outbox Pattern (reliable event publishing)

Problem: you must update the DB **and** publish an event atomically, but DB and broker are separate systems (dual-write problem). Solution: in the **same local transaction**, write the domain change and an **outbox** row. A separate

relay (polling or CDC via Debezium) reads the outbox and publishes to Kafka, then marks it sent. Guarantees at-least-once publish consistent with the DB.

```
@Transactional { orderRepo.save(order); outboxRepo.save(event); } // atomic
relay/CDC -> read outbox -> publish to Kafka -> mark sent
```

Idempotency

Because delivery is at-least-once, consumers **must be idempotent**: dedup on a business/message key (`idempotency-key`, `eventId`), use `INSERT ... ON CONFLICT`, upserts, or a processed-ids table. Essential for retries, saga steps, and payment APIs.

Interview Q / Follow-ups

- Why not 2PC in microservices?
- Saga: choreography vs orchestration — trade-offs.
- How do you publish events reliably? (*outbox / CDC — solves dual-write.*)
- How do you make a consumer idempotent?
- What is a compensating transaction? Example.

Hands-on Exercise

Design an order saga with orchestration + compensations; add an outbox table and a relay; make the payment consumer idempotent via an `idempotency_key` unique constraint.

7.6 Service Communication: REST vs gRPC & Event-Driven

Sync vs Async

- **Synchronous (REST/gRPC)** — request/response, temporal coupling; simpler, but the caller waits and failures propagate.
- **Asynchronous (events/messaging)** — publish events, consumers react; decoupled, resilient, scalable, but eventually consistent and harder to trace.

REST vs gRPC

	REST/JSON	gRPC
Transport	HTTP/1.1 (or 2)	HTTP/2
Payload	JSON (text)	Protobuf (binary, compact)
Contract	OpenAPI (loose)	<code>.proto</code> (strict, codegen)
Streaming	limited (SSE)	bidirectional streaming
Perf	good	higher throughput, lower latency
Browser	native	needs grpc-web
Use	public APIs, simplicity	internal high-perf service-to-service

Event-Driven Architecture (EDA)

Services emit domain events to a broker (Kafka); consumers react. Enables decoupling, replay, CQRS, event sourcing. Trade-off: eventual consistency, ordering, debugging across async flows.

Interview Q / Follow-ups

- REST vs gRPC — when each?
- Sync vs async communication trade-offs.
- What is EDA; benefits and challenges?

- How do you trace a request across async boundaries? (*correlation/trace id propagation* — Module 11.)

Module 7 — One-Page Cheat Sheet

Topic	Key point
Monolith vs micro	micro trades code coupling for distributed complexity; avoid distributed monolith
Gateway	north-south entry: routing/auth/rate-limit/aggregation
Discovery	Eureka/Consul/K8s DNS; client vs server-side
Config	Spring Cloud Config (Git) / ConfigMaps
Timeout	bound every remote call (first defense)
Retry	idempotent only; exp backoff + jitter; cap
Circuit breaker	CLOSED → OPEN → HALF_OPEN; fail fast + fallback
Bulkhead	isolate pools per dependency
Saga	local txns + compensations; choreography vs orchestration
Outbox	atomic DB write + event row; relay/CDC publishes (fixes dual-write)
Idempotency	dedup on key; at-least-once needs it
REST vs gRPC	JSON/HTTP1 vs protobuf/HTTP2 streaming

Module 7 — Top Interview Questions

1. How do you handle transactions across microservices? (saga, why not 2PC)
2. Explain circuit breaker states; combine with retry + timeout.
3. Outbox pattern and the dual-write problem.
4. Why must retried/consumed operations be idempotent; how to implement.
5. Choreography vs orchestration saga.
6. REST vs gRPC; sync vs async trade-offs.
7. API gateway responsibilities; gateway vs service mesh.
8. Service discovery mechanisms.
9. How to prevent retry storms / cascading failures.
10. How do you define service boundaries?

Module 7 — Common Mistakes

- Sharing a database across services (distributed monolith).
- Retrying non-idempotent operations.
- Dual-write without outbox (lost events).
- No timeouts → cascading failures / thread exhaustion.
- Circuit breaker without a fallback.

Module 7 — Mock Interview

1. “Order needs inventory + payment across services — how do you keep them consistent?” → orchestration saga with compensations; outbox for reliable events; idempotent consumers.

2. *"A downstream dependency is slow and taking your service down."* → timeouts + bulkhead + circuit breaker + fallback.
3. *"You publish an event after committing the DB, but sometimes the event is lost."* → dual-write; use outbox/CDC.
4. *"A consumer processed the same payment twice."* → at-least-once delivery; add idempotency key + unique constraint / dedup store.
5. *"Internal services need very low latency, high throughput."* → gRPC over protobuf/HTTP2; keep REST for public APIs.

Next → Module 8: Messaging (Kafka & RabbitMQ).